

M.S.P. Mandal's

Deogiri College, Chhatrapati Sambhajinagar

(An Autonomous institute affiliated to Dr. B.A.M. University, Chhatrapati Sambhajinagar)



A PROJECT REPORT ON

“Employee Preparation Platform”

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Science in Information Technology

Submitted By

1.RIYA SHIVAJI NAGARE

[Roll No.-34/Prn – 2425007356]

Under the Guidance of

Mr. Rushikesh S. Chandane

Assistant Professor, CS, IT & A

DEPARTMENT OF COMPUTER SCIENCE, I.T. & ANIMATION

DEOGIRI COLLEGE

CHHATRPTI SAMBHAJINAGAR -431005

[2026]

M.S.P. Mandal's

Deogiri College, Chhatrapati Sambhajinagar

(An Autonomous institute affiliated to Dr. B.A.M. University, Chhatrapati Sambhajinagar)

DEPARTMENT OF COMPUTER SCIENCE, I.T. & ANIMATION



NAAC Reaccredited with "A++" Grade

CERTIFICATE

This is to certify that **Ms. RIYA SHIVAJI NAGARE** has successfully completed the project entitled **"EMPLOYEE PREPARATION PLATFORM"** in the subject **"Python Programming"** (DSC-23/BIT/T/503) in B.Sc. (Information Technology), Semester V, during the academic year 2026–2027.

Hence, certified.

Seat No: - _____

Date: - ____/____/____

Project Guide
Mr. Rushikesh S. Chandane

Department Incharge
Mr. Bhushan S. Kulkarni

Head of Department
Dr. Ritesh A. Magre

INDEX

Sr. No.	Title / Contents	Page No.
1	Introduction	1
2	Problem Statement	2
3	Objectives	3
4	Scope of the Project	4
5	Project Description / Methodology	5
6	Technologies & Tools Used	6
7	System Requirements	7
8	Implementation & Working	8
9	Output / Screenshots	9
10	Advantages & Limitations	11
11	Future Scope	12
12	Conclusion	13
13	References	14
14	Appendix – Source Code	15

1. Introduction

In today's competitive job market, the ability to perform well in interviews has become a decisive factor in a candidate's career growth. Most job seekers, especially final-year students and working professionals switching jobs, lack access to structured, personalized, and affordable resources for interview preparation. Traditional methods such as books, coaching classes, and scattered online resources are often expensive, generic, and fail to adapt to an individual's specific strengths and weaknesses.

This project, titled the “AI-Based Interview Preparation Platform,” is a web application built using Python that brings together all the essential tools a job seeker needs under one roof. It combines resume building and analysis, aptitude and coding practice, company-specific mock tests, and AI-assisted feedback into a single, easy-to-use platform, helping candidates prepare more efficiently and track their progress over time.

2. Problem Statement

Job seekers today face several challenges while preparing for interviews and placement drives. Free resources available online are scattered across multiple websites, resume-building tools often require payment for advanced features, and there is no single platform that combines aptitude tests, coding practice, and interview preparation with instant, AI-based feedback. As a result, candidates spend excessive time switching between platforms instead of focusing on actual preparation.

There is a clear need for an integrated, free, and intelligent platform that can guide a candidate through every stage of interview preparation, from building a strong resume to practicing company-specific questions, while also providing meaningful, automated feedback.

3. Objectives

- To design and develop a unified web-based platform for end-to-end interview preparation.
- To build a secure authentication system using JWT tokens and password hashing.
- To provide a resume builder that generates a professional PDF resume, along with an AI-based content enhancement feature.
- To build a resume analyzer that extracts and evaluates the content of an uploaded resume.
- To create company-specific mock tests with automatic scoring.
- To provide a coding practice module covering multiple programming languages.
- To track and display a user's overall preparation progress through a dashboard.

4. Scope of the Project

The scope of this project covers the design and development of a web-based interview preparation system intended for students and working professionals preparing for placement interviews or job switches.

Included in Scope

- User registration and login with secure authentication.
- A dashboard providing access to all preparation modules.
- Resume Builder and Resume Analyzer modules.
- Company-specific mock test module with MCQ-based questions.
- Coding practice module for multiple programming languages.

Not Covered in the Current Scope

- Full video or voice-based proctored interviews.
- Support for languages other than English.
- Large-scale cloud deployment for public or commercial use.

5. Project Description / Methodology

The platform follows a client-server architecture. The backend is developed using Python's FastAPI framework, with SQLAlchemy used as the Object Relational Mapper (ORM) to interact with the database. The frontend is built using plain HTML, CSS, and JavaScript, which communicates with the backend through RESTful API calls.

Working Methodology

- A user registers and logs in; credentials are secured using bcrypt hashing, and the session is maintained using JWT tokens.
- After logging in, the user is directed to a dashboard containing cards for each module.
- In the Resume Builder, the user fills a dynamic form; the backend generates a downloadable PDF resume using the ReportLab library, and an “Enhance with AI” option refines the content using an AI language model.
- In the Resume Analyzer, an uploaded PDF resume is parsed using the pdfplumber library to extract text, word count, and contact details, and feedback is generated based on the extracted content.
- The Company Mock Test module presents multiple-choice questions specific to companies such as TCS, Infosys, and Google, with automatic scoring on submission.
- The Coding Practice module allows the user to select a programming language and practice coding questions in a simple code editor.

6. Technologies and Tools Used

Backend

- Python
- FastAPI (web framework)
- SQLAlchemy (ORM)

Frontend

- HTML5
- CSS3
- JavaScript

Database

- SQLite (development)
- MySQL (production)

Other Tools and Libraries

- JWT and bcrypt for authentication and password security
- ReportLab for PDF resume generation
- pdfplumber for PDF resume parsing
- An AI language model API for resume content enhancement
- Visual Studio Code as the development environment

7. System Requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB minimum (8 GB recommended)
- Hard Disk: 250 MB of free space

Software Requirements

- Operating System: Windows 10 / 11
- Python 3.10 or above
- Visual Studio Code (or any code editor)
- Web Browser: Google Chrome / Microsoft Edge
- Required Python packages: fastapi, uvicorn, sqlalchemy, pdfplumber, reportlab, python-jose, passlib[bcrypt]

8. Implementation and Working

The platform has been implemented module by module, with each module tested independently before integration into the complete system.

Authentication Module

Allows users to sign up and log in securely. Passwords are hashed using bcrypt before being stored, and a JWT token is issued on successful login to maintain the user's session.

Dashboard Module

Displays a welcome message along with module cards for Resume Builder, Resume Analyzer, Company Mock Test, and Coding Practice, giving the user quick access to every feature.

Resume Builder and Analyzer

The Resume Builder collects user details through a dynamic form and generates a formatted PDF resume. The Resume Analyzer accepts an uploaded resume and provides feedback on word count, missing sections, and contact information.

Company Mock Test Module

Presents company-specific multiple-choice questions and calculates the score automatically once the test is submitted.

9. Output / Screenshots

This section presents the key screens of the developed platform.

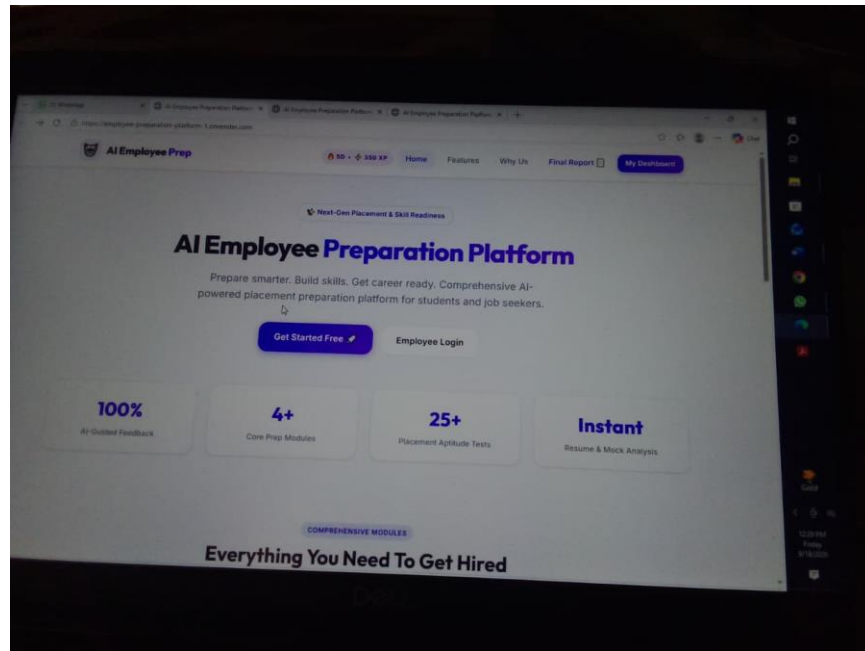


Figure 1: Platform Home Page – AI Employee Preparation Platform

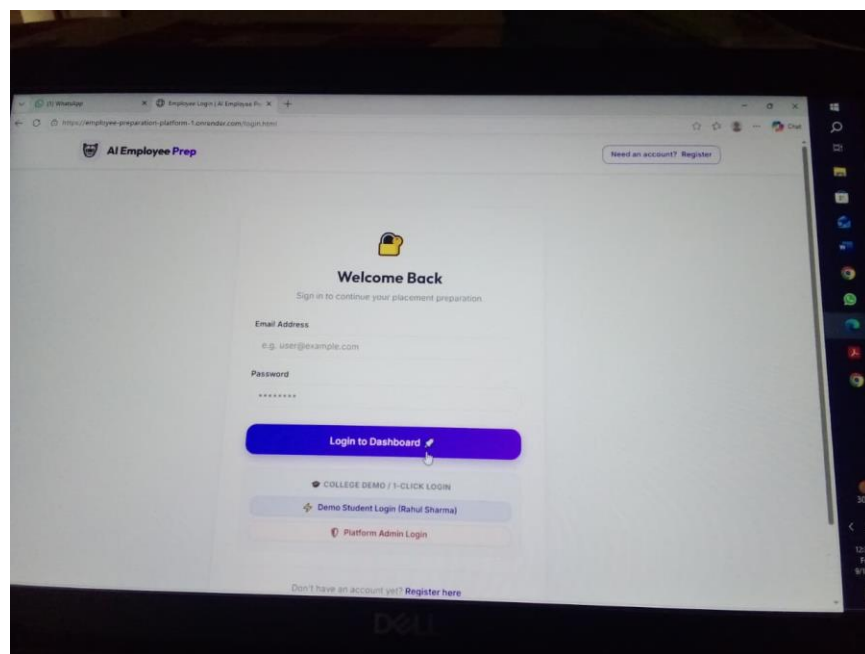


Figure 2: Employee Login Page

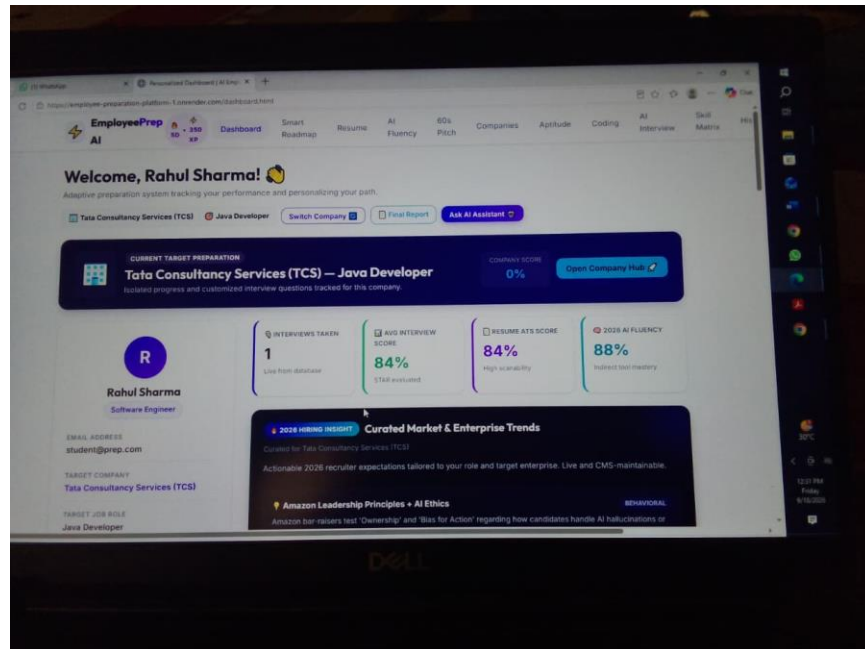


Figure 3: Personalized Dashboard – Overview and Company Score

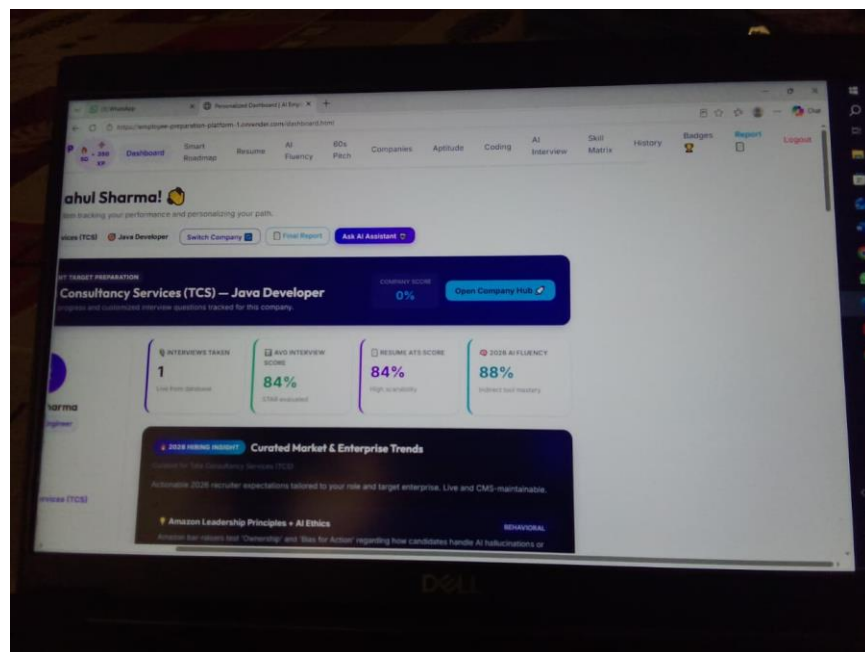


Figure 4: Dashboard – Navigation Menu and Hiring Insights

10. Advantages and Limitations

Advantages

- Combines multiple preparation tools into a single, free platform.
- Saves time by eliminating the need to switch between different websites.
- AI-based resume enhancement improves the quality of resume content.
- Company-specific mock tests increase placement readiness.
- Simple and user-friendly interface accessible to all students.

Limitations

- Requires an active internet connection for AI-based features.
- The company-specific question bank is currently limited to a few organizations.
- Coding practice currently uses simulated validation rather than a full code-execution engine.
- The platform is currently deployed locally and not yet hosted on the cloud.

11. Future Scope

- Integrating a voice and video-based AI mock interview module.
- Expanding the company-specific question bank to cover more organizations.
- Adding real-time code execution using a sandboxed judge engine.
- Deploying the platform on a cloud service for public access.
- Adding an analytics dashboard to track a user's improvement over time.
- Introducing additional modules such as salary negotiation and soft-skill practice.

12. Conclusion

The AI-Based Interview Preparation Platform successfully demonstrates how Python-based web technologies can be combined with AI capabilities to build a practical, integrated solution for interview preparation. By bringing resume building, resume analysis, coding practice, and company-specific mock tests together on one platform, the project addresses a genuine need faced by students and job-switchers. The current implementation lays a strong and scalable foundation that can be extended with more advanced AI-driven features in the future.

13. References

- FastAPI Documentation – <https://fastapi.tiangolo.com>
- SQLAlchemy Documentation – <https://www.sqlalchemy.org>
- Python Official Documentation – <https://docs.python.org>
- ReportLab User Guide – <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- pdfplumber Repository – <https://github.com/jsvine/pdfplumber>
- W3Schools Web Development Tutorials – <https://www.w3schools.com>

14. Appendix – Source Code

14.1 Database ORM Models (backend/models_sa.py)

```
from datetime import datetime
from sqlalchemy import create_engine, Column, Integer, String, Text, Float, DateTime,
ForeignKey
from sqlalchemy.orm import declarative_base, sessionmaker, relationship

DATABASE_URL = "sqlite:///backend/database/employees.db"
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

class User(Base):
    __tablename__ = "employees"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    name = Column(String(100), nullable=False)
    email = Column(String(120), unique=True, nullable=False, index=True)
    password = Column(String(255), nullable=False)
    qualification = Column(String(150), nullable=False)
    skills = Column(Text, nullable=False)
    job_role = Column(String(100), nullable=False)
    target_company = Column(String(150), default="Tata Consultancy Services (TCS)")
    target_role = Column(String(150), default="Software Engineer")
    created_at = Column(DateTime, default=datetime.utcnow)
    roadmaps = relationship("SmartRoadmap", back_populates="employee", cascade="all, delete-
orphan")
    resume_versions = relationship("ResumeVersion", back_populates="employee", cascade="all,
delete-orphan")

class SmartRoadmap(Base):
    __tablename__ = "smart_roadmaps"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    employee_id = Column(Integer, ForeignKey("employees.id", ondelete="CASCADE"),
nullable=False)
    company_name = Column(String(150), nullable=False)
    duration_days = Column(Integer, nullable=False)
    total_tasks = Column(Integer, default=0)
    completed_tasks = Column(Integer, default=0)
    progress_percent = Column(Float, default=0.0)
    streak_days = Column(Integer, default=1)
    tasks = relationship("RoadmapTask", back_populates="roadmap", cascade="all, delete-
orphan")

class ResumeVersion(Base):
    __tablename__ = "resume_versions"
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)
    employee_id = Column(Integer, ForeignKey("employees.id", ondelete="CASCADE"),
nullable=False)
    version_name = Column(String(150), nullable=False)
    template_name = Column(String(100), default="modern-single")
    resume_data_json = Column(Text, nullable=False)
    score = Column(Integer, default=0)
    updated_at = Column(DateTime, default=datetime.utcnow)
```

14.2 Application Server & API Dispatcher (backend/fastapi_app.py)

```
import os
```

```

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from backend.database import init_db
from backend.models import (
    DashboardSummaryModel, SmartRoadmapModel,
    TrendingTemplateModel, AIFluencyModel, StructuredAnswerModel
)

init_db()
app = FastAPI(title="AI Employee Preparation Platform API", version="2026.1")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/api/dashboard/summary")
def get_dashboard_summary(employee_id: int):
    summary = DashboardSummaryModel.get_summary(employee_id)
    return {"success": True, "data": summary}

@app.get("/api/roadmap/smart")
def get_smart_roadmap(employee_id: int, company: str = ""):
    roadmap = SmartRoadmapModel.get_current(employee_id, company)
    return {"success": True, "data": roadmap}

@app.post("/api/roadmap/smart/task/toggle")
async def toggle_smart_roadmap_task(request: Request):
    data = await request.json()
    result = SmartRoadmapModel.toggle_task(int(data.get("employee_id")),
    int(data.get("task_id")))
    return result

@app.get("/api/resume/templates")
def get_resume_templates():
    return {"success": True, "templates": TrendingTemplateModel.get_active()}

@app.post("/api/ai-fluency/evaluate")
async def evaluate_ai_fluency(request: Request):
    data = await request.json()
    return AIFluencyModel.evaluate_answer(
        int(data.get("employee_id")),
        data.get("question_id"),
        data.get("candidate_answer", "")
    )

@app.post("/api/answer-builder/analyze")
async def analyze_structured_pitch(request: Request):
    data = await request.json()
    return StructuredAnswerModel.analyze(
        employee_id=int(data.get("employee_id", 1)),
        question_title=data.get("question_title", "Tell me about yourself"),
        present_text=data.get("present_text", ""),
        past_text=data.get("past_text", ""),
        future_text=data.get("future_text", "")
    )

```

)

14.3 Authentication & Session Security (backend/routes/auth.py)

```
import hashlib, secrets
from flask import Blueprint, request, jsonify
from backend.database import get_db

auth_bp = Blueprint("auth", __name__)

def hash_password(password: str) -> str:
    return hashlib.sha256(password.encode("utf-8")).hexdigest()

@auth_bp.route("/api/auth/register", methods=["POST"])
def register():
    data = request.get_json() or {}
    name, email, password = data.get("name"), data.get("email"), data.get("password")
    if not name or not email or not password:
        return jsonify({"success": False, "message": "All fields required"}), 400

    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("SELECT id FROM employees WHERE email = ?", (email,))
    if cursor.fetchone():
        return jsonify({"success": False, "message": "Email already registered"}), 409

    cursor.execute(
        """INSERT INTO employees (name, email, password, qualification, skills, experience,
job_role)
VALUES (?, ?, ?, ?, ?, ?, ?)""",
        (name, email, hash_password(password), data.get("qualification", "B.Sc IT"),
data.get("skills", "Python, SQL"), data.get("experience", "Fresher"),
data.get("job_role", "Software Engineer"))
    )
    conn.commit()
    return jsonify({"success": True, "message": "Registration successful"}), 201

@auth_bp.route("/api/auth/login", methods=["POST"])
def login():
    data = request.get_json() or {}
    email, password = data.get("email"), data.get("password")
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("SELECT id, name, email, password, is_admin FROM employees WHERE email =
?", (email,))
    user = cursor.fetchone()
    if not user or user["password"] != hash_password(password):
        return jsonify({"success": False, "message": "Invalid email or password"}), 401

    session_token = secrets.token_hex(24)
    return jsonify({
        "success": True,
        "token": session_token,
        "user": {"id": user["id"], "name": user["name"], "email": user["email"], "is_admin":
user["is_admin"]}
    }), 200
```

14.4 2026 AI Fluency Round Engine (backend/routes/ai_fluency.py)

```

from flask import Blueprint, request, jsonify
from backend.models import AIFluencyModel

ai_fluency_bp = Blueprint("ai_fluency", __name__)

@ai_fluency_bp.route("/api/ai-fluency/questions", methods=["GET"])
def get_ai_fluency_questions():
    questions = AIFluencyModel.get_questions(request.args.get("category", ""))
    return jsonify({"success": True, "questions": questions}), 200

@ai_fluency_bp.route("/api/ai-fluency/evaluate", methods=["POST"])
def evaluate_ai_fluency():
    """
    Evaluates candidate response across 4 core dimensions:
    - Tool Integration (25%)
    - Critical Verification & Security (35%)
    - Velocity & Sprint Impact (20%)
    - Authenticity & Communication (20%)
    """
    data = request.get_json() or {}
    employee_id = data.get("employee_id")
    if not employee_id:
        return jsonify({"success": False, "message": "employee_id is required"}), 400

    result = AIFluencyModel.evaluate_answer(
        int(employee_id),
        data.get("question_id"),
        data.get("candidate_answer", "")
    )
    return jsonify(result), (200 if result.get("success") else 400)

```

14.5 Present-Past-Future 60s Pitch Builder (backend/routes/answer_builder.py)

```

from flask import Blueprint, request, jsonify
from backend.models import StructuredAnswerModel

answer_builder_bp = Blueprint("answer_builder", __name__)

@answer_builder_bp.route("/api/answer-builder/analyze", methods=["POST"])
def analyze_structured_answer():
    """
    Analyzes Present-Past-Future pitch for speaking pace (140 WPM target, 50-70s optimal).
    """
    data = request.get_json() or {}
    analysis = StructuredAnswerModel.analyze(
        employee_id=int(data.get("employee_id", 1)),
        question_title=data.get("question_title", "Tell me about yourself"),
        present_text=data.get("present_text", ""),
        past_text=data.get("past_text", ""),
        future_text=data.get("future_text", ""),
        target_role=data.get("target_role", "Software Engineer"),
        target_company=data.get("target_company", "Enterprise")
    )
    return jsonify(analysis), 200

@answer_builder_bp.route("/api/answer-builder/save", methods=["POST"])
def save_structured_answer():
    data = request.get_json() or {}
    result = StructuredAnswerModel.save(int(data.get("employee_id")), data)

```

```
return jsonify(result), 200
```

14.6 Frontend REST API Controller (frontend/js/api.js)

```
const API_BASE = window.location.origin;
```

```
const API = {  
  getUser: () => JSON.parse(localStorage.getItem('currentUser')) || null,  
  setUser: (user) => localStorage.setItem('currentUser', JSON.stringify(user)),  
  logout: () => { localStorage.removeItem('currentUser'); window.location.href = 'login.html';  
},
```

```
  request: async (endpoint, options = {}) => {  
    try {  
      const res = await fetch(`${API_BASE}${endpoint}`, {  
        ...options,  
        headers: { 'Content-Type': 'application/json', ...options.headers }  
      });  
      return await res.json();  
    } catch (err) {  
      console.error(`API Error [${endpoint}]:`, err);  
      return { success: false, message: 'Network connection failure.' };  
    }  
  },
```

```
  login: (email, password) => API.request('/api/auth/login', {  
    method: 'POST',  
    body: JSON.stringify({ email, password })  
  }),
```

```
  getDashboardSummary: (empId) => API.request(`/api/dashboard/summary?employee_id=${empId}`),  
  getSmartRoadmap: (empId) => API.request(`/api/roadmap/smart?employee_id=${empId}`),  
  toggleRoadmapTask: (taskId, empId) => API.request('/api/roadmap/smart/task/toggle', {  
    method: 'POST',  
    body: JSON.stringify({ task_id: taskId, employee_id: empId })  
  }),  
  evaluateAIFluency: (empId, qId, ans) => API.request('/api/ai-fluency/evaluate', {  
    method: 'POST',  
    body: JSON.stringify({ employee_id: empId, question_id: qId, candidate_answer: ans })  
  })  
};
```